STEP 2 · 2주차 제출물

전처리와 특징 추출
원신호 → 2초 → 19 × 200ms → 25차원

60Hz 노치 + 20–450Hz 대역통과(filtfilt) → 2초 세그먼트(0.7~2.7s) →
200ms·50% 윈도우 → RMS·MAV·WL·ZC·SSC × 5채널 → 시행 단위 8:2 분할 → 표준화

① 필터 검증

② 윈도우 확인

③ 특징 구현 코드

④ 분할 코드

⑤ 데이터셋 요약

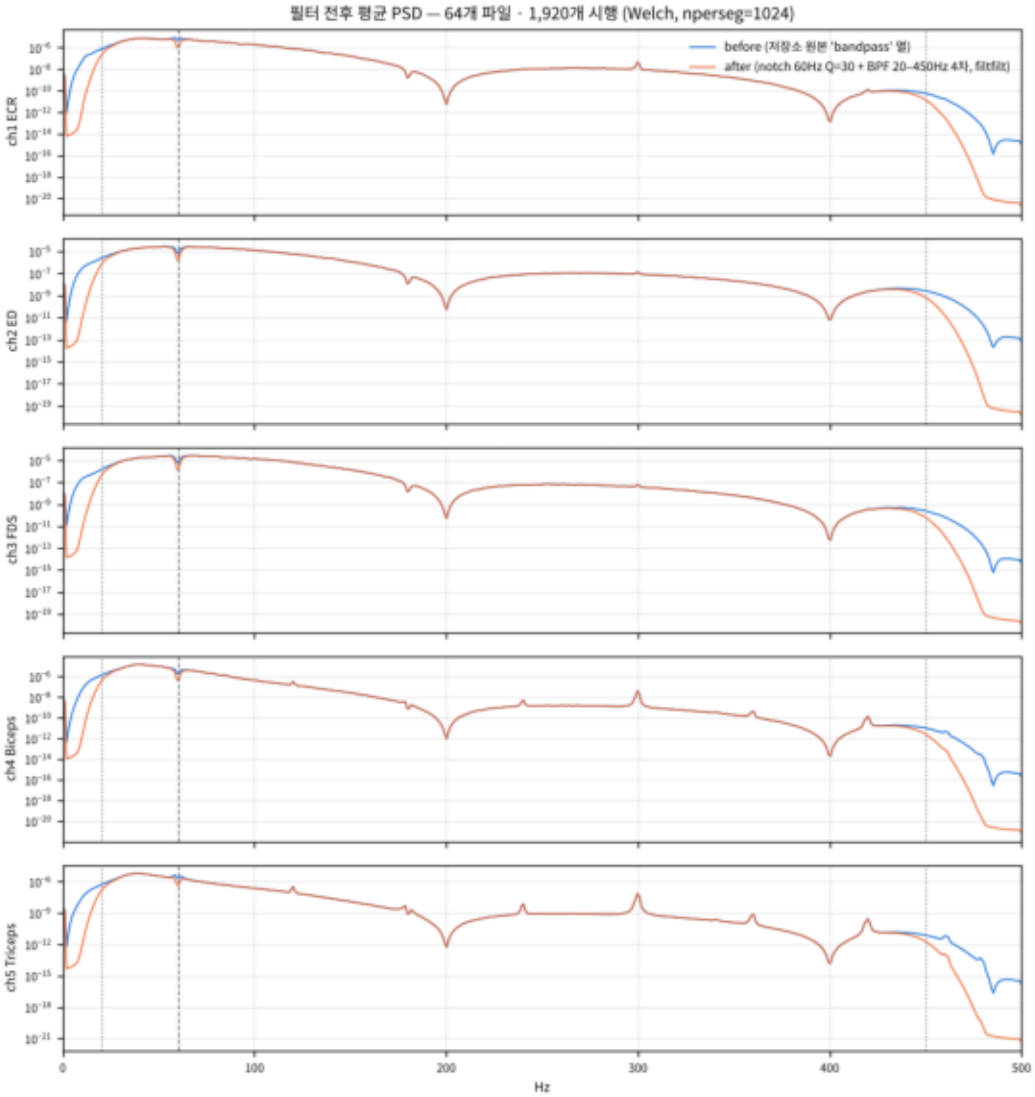
⑥ 설정 근거

학번 19146809 · 컴퓨터공학과 · 김지훈

논문: Simultaneous Classification of Hand Gestures and Elbow Angles Using sEMG Signals:
A Comparison of Machine Learning Performances — Biomedical Engineering Letters (2026)
코드: github.com/ddeeqq/learning_pj2 · 데이터셋 파일: results/week2/dataset.npz

① 필터 검증

filter_check.png — 필터 전후 스펙트럼 비교 (실습 단계 1)



슬라이드 9쪽 preprocess()를 그대로 썼다(노치 Q=30, 4차 Butterworth 20–450Hz, filtfilt, axis=0). 파일 하나(30회 연속 기록)를 통째로 거른 뒤 시행을 잘라, 5초 조각 양 끝의 필터 가장자리 효과를 피했다.

- 20Hz 아래(1–15Hz) 평균 전력이 채널별로 326~486배, 450Hz 위(460–500Hz)는 117~163배 줄었다. 대역통과의 양쪽 경계가 제대로 걸렸다.
- 근활성 대역(30–400Hz)의 전력은 1.04~1.09배로 거의 그대로다 — 필요한 신호는 건드리지 않았다.
- 60Hz 지점 전력은 4.2~4.9배 더 낮아졌다. 다만 before 곡선에도 60Hz 봉우리가 없고 오히려 홈이 파여 있다. 저장소의 'bandpass' 열이 이미 노치 + 20–450Hz 대역통과를 거친 신호이기 때문이다(1주차 확인). 그래서 이 그림은 잡음이 사라지는 장면이 아니라, 같은 필터를 한 번 더 걸어도 통과 대역은 유지되고 대역 밖만 추가로 눌린다는 확인으로 읽어야 한다.
- 원 데이터에는 180·200·400Hz 부근에도 좁은 홈이 있다(저장소 측 처리로 추정). 우리 필터와는 무관하다.

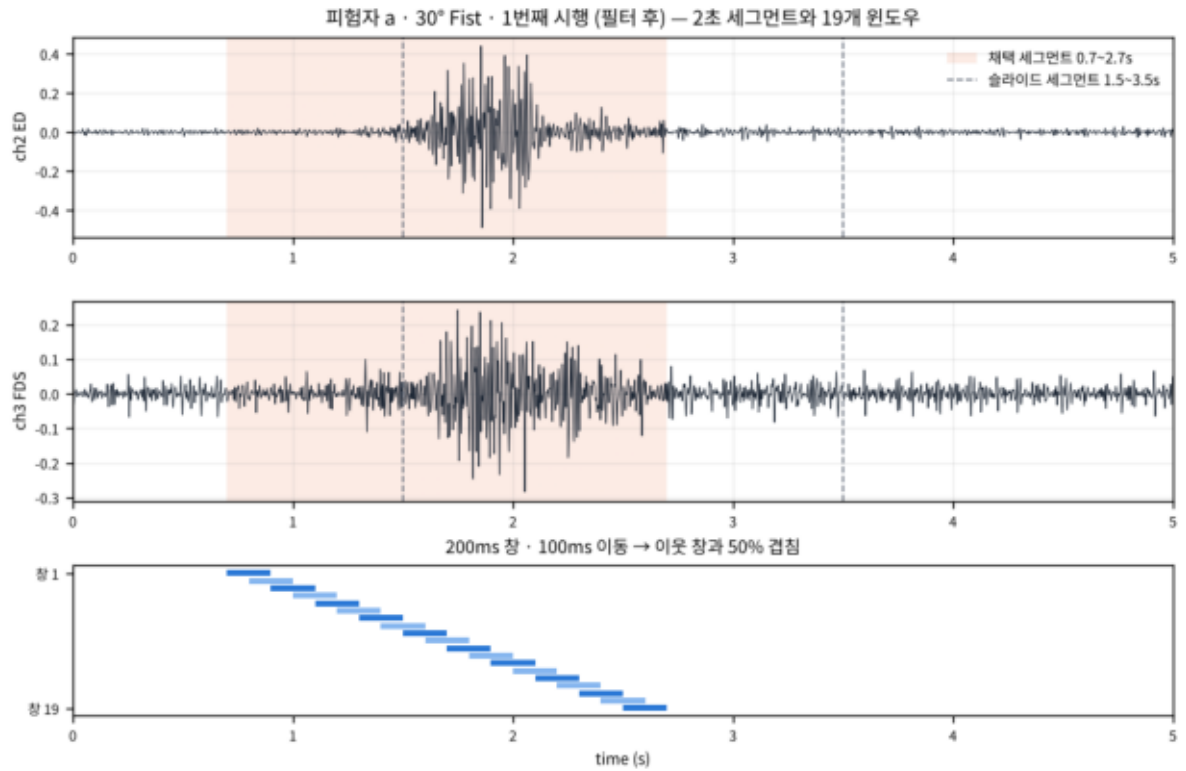
② 윈도우 확인

윈도우 배열 모양 출력 화면 (실습 단계 1~3)

```
$ python src/week2_lab.py
[단계 1] 필터 적용 — a_30 extor.csv
전: [0.0103 0.0141 0.0117 0.0202 0.0048]
후: [0.0089 0.0136 0.0113 0.0193 0.0042]

[단계 2] 분석 구간 — START, LEN = 700 2000
세그먼트 모양: (2000, 5)

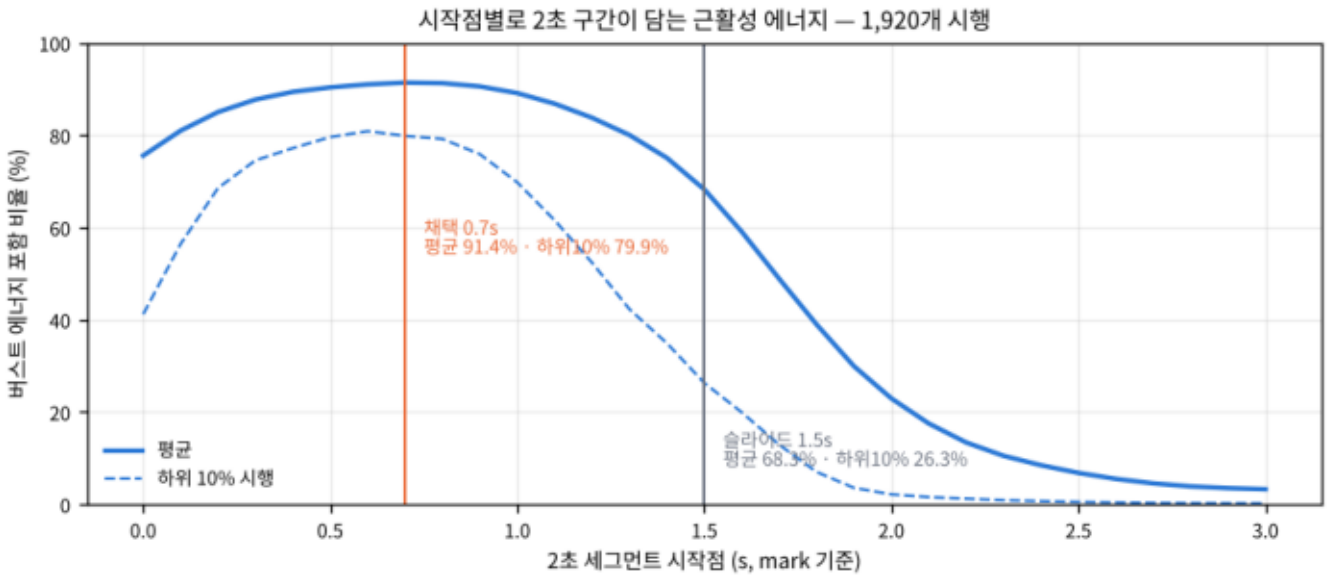
[단계 3] 슬라이딩 윈도우 — WIN, HOP = 200 100
윈도우 배열: (19, 200, 5)
19 = (2000 - 200) / 100 + 1 = 19
```



- 필터 후 5채널 표준편차가 출력되고, 세그먼트 (2000, 5) → 윈도우 (19, 200, 5)로 슬라이드의 기대값과 같다. 19 = (2000 − 200) / 100 + 1.
- 세그먼트 시작점은 슬라이드의 실습 가정(1.5초) 대신 0.7초를 썼다. 근거는 다음 쪽.
- 아래 막대처럼 이웃 창은 100샘플씩 겹친다. 그래서 같은 시행의 창이 학습·테스트 양쪽에 들어가면 누수가 된다(④).

② 윈도우 확인 (계속)

2초 세그먼트 시작점 — 슬라이드 10쪽 '데이터에 맞게 조정'



슬라이드는 1.5초 시작을 '실습 가정이며 논문 설정이 아님'이라고 밝히고, 지난주 그림에서 동작 구간을 확인해 조정하라고 한다. 1주차에서 이 데이터의 근활성 최대 시점은 mark 기준 중앙값 1.69초(10~90%: 1.21~2.07초)로, 슬라이드가 가정한 '2~3초 동작'보다 앞섰다.
그래서 시작점을 0~3초까지 0.1초 간격으로 옮겨 가며, 2초 구간이 각 시행의 버스트 에너지(마지막 1초 휴식을 기준선으로 뺀 값)를 몇 % 담는지 1,920개 시행 전체에서 계산했다.

시작점	구간	에너지 포함 (평균)	에너지 포함 (하위 10% 시행)
1.5s (슬라이드)	1.5 ~ 3.5s	68.3%	26.3%
0.7s (채택)	0.7 ~ 2.7s	91.4%	79.9%

- 평균이 가장 큰 시작점이 0.7초(91.4%)였고 이를 채택했다. 슬라이드 기본값을 그대로 쓰면 평균 68%만 담기고, 하위 10% 시행은 26%로 동작 대부분을 놓친다 — 19개 창 중 상당수가 휴식 구간이 되어 클래스 정보가 없는 표본이 늘어난다.
- 이 선택은 라벨이 아니라 신호 에너지만 보고 정했고, 8클래스 모두에 같은 시작점을 쓰므로 분류 결과를 미리 보고 고른 값은 아니다.

③ 특징 구현 코드

td_features 함수 전체 — src/week2_lab.py (numpy만 사용)

```
241  def td_features(w, th=ZC_TH):
242      """w: (200, 5) -> (25,)
243      순서: RMS×5, MAV×5, WL×5, ZC×5, SSC×5  (채널: ECR, ED, FDS, Biceps, Triceps)"""
244      rms = np.sqrt(np.mean(w ** 2, axis=0))
245      mav = np.mean(np.abs(w), axis=0)
246      d = np.diff(w, axis=0)
247      wl = np.sum(np.abs(d), axis=0)
248      zc = np.sum(((w[:-1] * w[1:]) < 0) &
249                  (np.abs(d) >= th), axis=0)
250      ssc = np.sum((d[:-1] * d[1:]) < 0, axis=0)
251      return np.concatenate([rms, mav, wl, zc, ssc])
```

25차원 순서 (리포트·다음 주 해석용)

열 번호	특징	채널 순서	식
0–4	RMS	ECR · ED · FDS · Biceps · Triceps	$\sqrt{(\text{mean } x^2)}$ — 신호 파워
5–9	MAV	ECR · ED · FDS · Biceps · Triceps	mean $\lvert x \rvert$
10–14	WL	ECR · ED · FDS · Biceps · Triceps	$\Sigma \lvert x[i+1] - x[i] \rvert$
15–19	ZC	ECR · ED · FDS · Biceps · Triceps	부호 바뀜 & $\lvert$차이$\rvert \geq 0.01$
20–24	SSC	ECR · ED · FDS · Biceps · Triceps	기울기 부호 바뀜 횟수

ZC 임계값 관찰 — th를 0으로 바꿔 보기 (슬라이드 12쪽)

th	ECR	ED	FDS	Biceps	Triceps	ZC=0 창 비율
0	25.8	28.4	28.9	22.1	23.3	0.0%
0.001	23.7	26.8	26.7	19.5	20.1	0.1%
0.005	13.2	18.0	18.1	9.8	7.9	10.5%
0.01	6.9	12.6	12.3	4.6	3.0	28.1%

표: 36,480개 창의 채널별 평균 ZC. th=0이면 창당 22~29회지만 th=0.01이면 3~13회로 줄고, 채널-창 중 28%는 ZC가 0이 된다. 이 데이터의 인접 샘플 차이는 중앙값 0.0029, 90% 지점 0.0166라서 0.01은 '잡음 제거'를 넘어 휴식 구간의 교차를 거의 다 지운다. 그 결과 ZC가 진폭(근수축 강도)과 비슷한 정보를 담게 된다. 이번에는 슬라이드·논문 조건(0.01)을 그대로 쓰고, 이 값이 신호 단위에 비해 크다는 점을 기록해 둔다.

④ 분할 코드

시행 단위로 먼저 나눈 부분 — 노란 줄이 분할 (src/week2_lab.py)

```
288  def split_trials_8_2(trials):
289      # ===== [분할] 시행 단위로 먼저 8:2 분할 — 윈도우·특징은 이 다음에 만든다 =====
290      tr_idx, te_idx = train_test_split(
291          np.arange(len(trials)), test_size=0.2,
292          stratify=trials["group"],          # 피험자×클래스(64그룹)마다 30회 중 24:6
293          random_state=SEED)
294      return trials.iloc[tr_idx].reset_index(drop=True), trials.iloc[te_idx].reset_index(drop=True)
```

```
297  def build(trial_rows, filt):
298      """주어진 시행들 안에서만 세그먼트 -> 윈도우 -> 특징을 만든다."""
299      X, y, tid, sub = [], [], [], []
300      for r in trial_rows.itertuples():
301          x = filt[W1.parse_name(r.file)][r.trial]          # (5000, 5) 필터 후 시행
302          for wnd in make_windows(cut(x)):
303              X.append(td_features(wnd)); y.append(r.label)
304              tid.append(f"{r.group}#{r.trial:02d}"); sub.append(r.subject)
305      return np.stack(X), np.array(y), np.array(tid), np.array(sub)
```

```
308  def step5_dataset(files, filt):
309      trials = trial_table(files)
310      tr_t, te_t = split_trials_8_2(trials)
311      Xtr, ytr, ttr, str_ = build(tr_t, filt)
312      Xte, yte, tte, ste = build(te_t, filt)
313      overlap = len(set(ttr) & set(tte))
```

- 분할 대상은 윈도우가 아니라 시행 번호 np.arange(1920)이다. 나눈 뒤에야 build()가 각 집합의 시행 안에서만 세그먼트 → 19개 창 → 특징을 만든다. 한 시행의 19개 창은 항상 같은 쪽에만 있다.
- 슬라이드는 파일 하나 = 시행 하나를 가정하고 files를 나누지만, 이 데이터는 파일 하나에 30회가 들어 있다. 파일 단위로 나누면 일부 '피험자×클래스' 조합이 통째로 테스트로 빠지므로, 시행 단위로 나누고 stratify를 피험자×클래스(64그룹)로 걸어 그룹마다 정확히 24:6이 되게 했다.

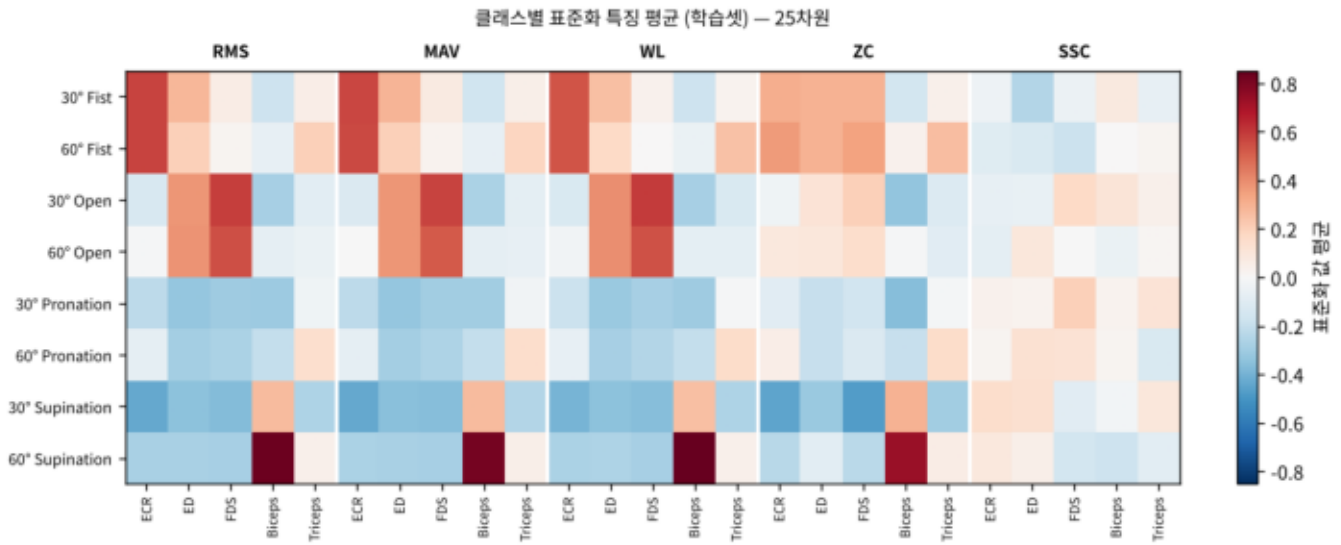
누수 점검	결과
학습 ∩ 테스트 시행 수	0 (겹치는 시행 없음)
학습 / 테스트 시행	1,536 / 384
표준화 통계	학습셋에서만 fit, 테스트셋은 transform (단계 6)

⑤ 데이터셋 요약

학습·테스트 샘플 수 · 클래스별 분포 · 표준화 확인 (단계 5·6)

라벨	클래스	학습 시행	학습 샘플(창)	테스트 시행	테스트 샘플(창)
30_fist	30° Fist	192	3,648	48	912
60_fist	60° Fist	192	3,648	48	912
30_open	30° Open	192	3,648	48	912
60_open	60° Open	192	3,648	48	912
30_pro	30° Pronation	192	3,648	48	912
60_pro	60° Pronation	192	3,648	48	912
30_extor	30° Supination	192	3,648	48	912
60_extor	60° Supination	192	3,648	48	912
합계	8클래스	1,536	29,184	384	7,296

항목	값
Xtr / Xte 모양	(29184, 25) / (7296, 25) — 학습이 테스트의 4.0배
피험자별 시행	8명 모두 학습 192 · 테스트 48 (같은 사람이 양쪽에 있음 — 아래 참고)
학습셋 표준화 후	평균 |최댓값| 1.1e-14 · 표준편차 1.000000~1.000000
테스트셋 표준화 후	평균 |최댓값| 0.031 · 표준편차 0.964~1.041 (0·1이 아니어도 정상)
상수 특징 / NaN	0개 / 0개 — 25개 모두 표준화 가능
dataset.npz	Xtr, ytr, Xte, yte + 시행 ID · 피험자 · 특징 이름 · 스케일러 통계 (5.1MB)



- 표준화된 특징의 클래스 평균을 보면 1주차의 관찰이 그대로 남아 있다: Fist는 전완 3채널, Open은 ED·FDS, Supination은 Biceps가 높고(60°에서 더 강함), Pronation은 전반적으로 낮다.
- RMS·MAV·WL 세 블록은 거의 같은 무늬다(모두 진폭 계열). 25차원이지만 서로 강하게 상관된 특징이 많다는 뜻이고, 다음 주 모델 비교 때 해석에 참고한다.
- 이 분할은 같은 피험자의 다른 시행이 양쪽에 들어가는 '피험자 내' 평가다. 처음 보는 사람에 대한 성능은 따로 피험자 단위 분할로 확인해야 한다.

⑥ 설정 근거

윈도우 200ms · 오버랩 50%를 고른 이유 (각 두 문장)

윈도우 200ms

sEMG는 수백 ms 안에서는 통계적 성질이 거의 변하지 않으므로(준정상성) 200ms 조각 하나는 안정된 신호로 다룰 수 있고, 그 안에 20Hz 성분도 4주기가 들어가 RMS·ZC 같은 통계량이 흔들리지 않을 만큼 샘플(200개)이 충분하다. 동시에 근전도 기반 제어에서 허용되는 지연(약 300ms) 안에 들어가므로, 실시간 기기 제어를 목표로 하는 이 문제의 조건과 맞는다.

오버랩 50% (100ms 이동)

100ms씩 밀면 2초 세그먼트 하나에서 겹침 없이 자를 때(10개)의 약 두 배인 19개 표본이 나와, 시행 수가 고정된 데이터에서 학습 표본을 늘리고 동작이 시작·끝나는 경계도 놓치지 않는다. 또 실제 사용 시 100ms마다 새 판단을 낼 수 있어 반응이 빨라지며, 겹친 창끼리 비슷하다는 대가는 시행 단위 분할(④)로 누수가 되지 않게 막았다.

함께 정한 설정

설정	값	근거
필터	노치 60Hz(Q=30) + BPF 20–450Hz 4차	논문 조건, Q·filtfilt는 실습 설정 (①)
세그먼트	2초, 0.7~2.7s	버스트 에너지 91% 포함, 슬라이드값 68% (②)
ZC 임계값	0.01	슬라이드·논문 조건 유지, 신호 대비 크다는 점 기록 (③)
분할	시행 단위 8:2, 피험자×클래스 층화	누수 방지 + 그룹마다 24:6 (④)
표준화	StandardScaler, 학습셋 fit	테스트 통계 누출 방지 (⑤)
시드	random_state = 42	다음 주에도 같은 분할

부록 · 점검표와 재현 방법

강의자료 15쪽 체크포인트

체크포인트	결과
필터 적용 후 5채널 표준편차 출력	완료 — ② 출력 화면
세그먼트가 (2000, 5)로 잘림	완료
윈도우 배열이 (19, 200, 5)	완료
특징 벡터가 (25,)	완료 — NaN 0개
시행 단위로 먼저 분할한 뒤 특징 생성	완료 — ④, 겹치는 시행 0
학습셋 평균 0 · 분산 1 확인	완료 — ⑤
dataset.npz 저장	완료 — results/week2/dataset.npz (5.1MB)

1주차 대비 달라진 점 · 기록

- 세그먼트 시작점: 1주차 관찰(활동 1.0~2.5s)을 반영해 0.7s로 조정.
- 분할 단위: 파일이 아니라 시행(1,920개). 파일 하나 = 30회라는 1주차 확인 결과에 따름.
- 저장 형식: np.savez_compressed — np.load로 그대로 읽힌다(키는 슬라이드와 같은 Xtr·ytr·Xte·yte).

재현 방법

```
$ pip install -r requirements.txt
$ git clone https://github.com/TestUser-alt-dev/sEMG-Hand-Gesture-Elbow-Angle-Dataset \
      data/sEMG-Hand-Gesture-Elbow-Angle-Dataset
$ python src/week2_lab.py               # 단계 1~6, results/week2/ 생성 (약 30초)
$ python report/generate_week2_report.py  # 이 PDF 생성

>>> d = np.load('results/week2/dataset.npz')
>>> d['Xtr'].shape, d['Xte'].shape      # (29184, 25) (7296, 25)
```